

# ALTR Model User Guide

This guide covers installing and running ALTR Model end-to-end, from raw input data to results and plots.

## [ALTR Model User Guide](#)

### [1. Install and Run](#)

[Quick start with Docker](#)

[Developer installation with uv](#)

[macOS / Linux](#)

[Windows \(PowerShell\)](#)

[Running the model](#)

[Standard Kedro run](#)

[Notebook](#)

[Batch runs without a notebook](#)

### [2. Input data dictionaries](#)

[assets\\_forecasts.csv](#)

[companies\\_ownerships.csv](#)

[scenarios.csv](#)

### [4. Pipeline reference](#)

[1. Inputs preparation](#)

[2. Shock mechanism](#)

[3. Staggered shock](#)

[4. Earnings model](#)

[5. Valuation model](#)

[6. Reporting plots](#)

### [5. Kedro quickstart](#)

### [7. Additional notes](#)

[Conditions a scenario has to respect to be economically viable](#)

[How assets get matched to a scenario geography](#)

[Granularity changes the shape of every output](#)

[Asset retirement age: refurbishment wrap-around, not a hard cutoff](#)

[Synthetic assets for increasing technologies](#)

# 1. Install and Run

## Quick start with Docker

1. Install [Docker Desktop](#).
2. Download the project: on the [GitHub repo page](#), click the green **Code** button → **Download ZIP**, then unzip it anywhere on your computer.
3. Put the three input CSVs (see [Input Data](#) below) in the `data/05_model_input/` folder inside the unzipped project.
4. Open a terminal in the unzipped folder and run:

Shell

```
docker compose up --build
```

5. Once the logs settle, open <http://localhost:8501> in your browser to open the batch-runner app.

## Developer installation with uv

We use [uv](#) for both installing Python and managing the project's virtual environment.

### macOS / Linux

Shell

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

```
git clone https://github.com/Theia-Finance-Labs/altr-model
cd altr-model
```

```
uv sync
source .venv/bin/activate
```

### Windows (PowerShell)

Shell

```
powershell -ExecutionPolicy Bypass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

```
git clone https://github.com/Theia-Finance-Labs/altr-model
cd altr-model
```

```
uv sync
.venv\Scripts\activate
```

## Running the model

ALTR runs on [Kedro](#) for orchestration — tags, `kedro run`, and the batch runners below all build on it. If you want the underlying concepts (nodes, catalog, parameters, environments) first, jump to [Kedro quickstart](#).

### Standard Kedro run

Use `--tags` to run the model stages as a group:

- `altrisk`: the actual model — inputs through valuation (NPV). This is the minimum to get results.
- `reporting`: produces plots from `altrisk` outputs. Depends on `altrisk` having already run.

Shell

```
# results only
```

```
kedro run --tags=altrisk
```

```
# results + plots
```

```
kedro run --tags=altrisk,reporting
```

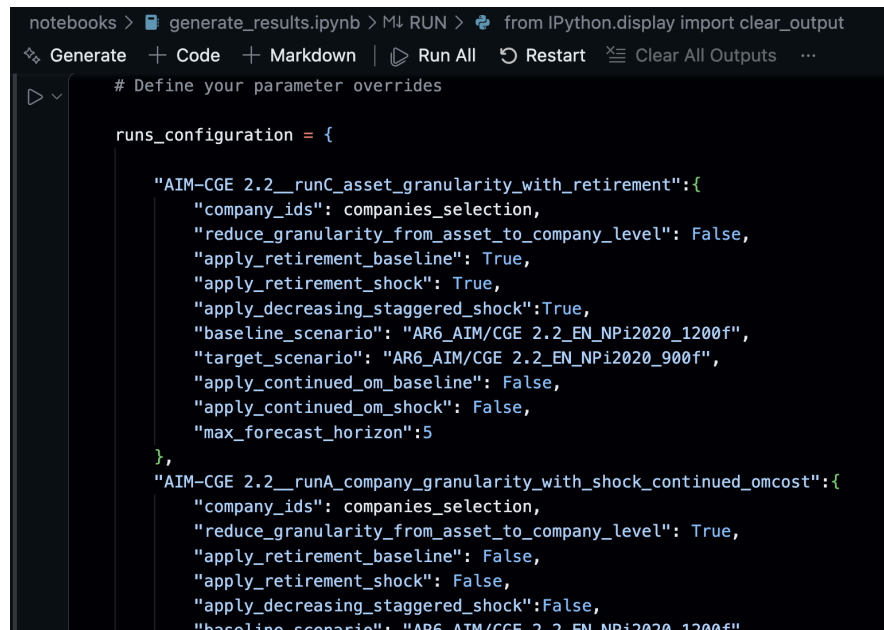
The configuration used is the one defined in `conf/base/`.

## Notebook

`notebooks/generate_results.ipynb` runs multiple parameterized configs in one go — useful for comparing scenarios or granularities without editing `conf/base/` between runs.

Two variables are used to control the content of the outputs:

- `runs_configuration`: a dict of `{run_name: {param_overrides}}`. Each entry becomes one `kedro run` with those parameters overridden on top of `conf/base/`.
- `companies_selection`: List of companies entering the modelisation.



```
notebooks > generate_results.ipynb > M RUN > from IPython.display import clear_output
Generate + Code + Markdown | Run All Restart Clear All Outputs ...

# Define your parameter overrides

runs_configuration = {

    "AIM-CGE 2.2__runC_asset_granularity_with_retirement":{
        "company_ids": companies_selection,
        "reduce_granularity_from_asset_to_company_level": False,
        "apply_retirement_baseline": True,
        "apply_retirement_shock": True,
        "apply_decreasing_staggered_shock":True,
        "baseline_scenario": "AR6_AIM/CGE 2.2_EN_NPi2020_1200f",
        "target_scenario": "AR6_AIM/CGE 2.2_EN_NPi2020_900f",
        "apply_continued_om_baseline": False,
        "apply_continued_om_shock": False,
        "max_forecast_horizon":5
    },
    "AIM-CGE 2.2__runA_company_granularity_with_shock_continued_omcost":{
        "company_ids": companies_selection,
        "reduce_granularity_from_asset_to_company_level": True,
        "apply_retirement_baseline": False,
        "apply_retirement_shock": False,
        "apply_decreasing_staggered_shock":False,
        "baseline_scenario": "AR6_AIM/CGE 2.2_EN_NPi2020_1200f"
```

## Batch runs without a notebook

`notebooks/run_kedro_batch.py` is the script form of the same pattern — useful for headless/CI-style batch runs, or as the engine behind the Streamlit app below:

Shell

```
python notebooks/run_kedro_batch.py \
    --run-configurations notebooks/example_run_configurations.yml \
    --company-ids notebooks/example_company_selection.csv \
    --output-dir workspace/results_batch \
    --tags altrisk
```

- `--run-configurations`: a YAML/JSON file shaped like the notebook's `runs_configuration` dict. `notebooks/example_run_configurations.yml` reproduces the six example configs from `generate_results.ipynb`.

- `--company-ids`: a CSV (`company_id` column), plain text (one id per line), or YAML/JSON list, applied to every run in the file.  
`notebooks/example_company_selection.csv` is a sample of 30 large companies picked to cover all four alignment × carbon-intensity quadrants (`aligned/misaligned` × `high_carbon/low_carbon`).
- Same output-copying behavior as the notebook, plus a `run_manifest.csv` per batch summarizing which runs succeeded.

## 2. Input data dictionaries

The model needs exactly three CSV files, placed in `data/05_model_input/`:

### `assets_forecasts.csv`

Physical power-generation assets (plants) and their technical characteristics — capacity, technology, location, age — one row per asset per forecast year.

| Column                     | Type   | Meaning                                            |
|----------------------------|--------|----------------------------------------------------|
| <code>asset_id</code>      | string | Unique identifier for the physical asset.          |
| <code>year</code>          | int    | Year this row's forecast values apply to.          |
| <code>sector</code>        | string | Economic sector the asset belongs to.              |
| <code>technology</code>    | string | Technology the asset uses within its sector.       |
| <code>capacity_unit</code> | string | Unit that <code>capacity</code> is expressed in.   |
| <code>capacity</code>      | float  | Production capacity of the asset in that year.     |
| <code>asset_age</code>     | float  | Age of the asset, in years, as of that row's year. |
| <code>country_iso2</code>  | string | ISO 3166-1 alpha-2 code of the asset's country.    |
| <code>country_name</code>  | string | Human-readable name of the asset's country.        |
| <code>asset_name</code>    | string | Human-readable name of the asset.                  |
| <code>latitude</code>      | float  | Asset location, latitude.                          |
| <code>longitude</code>     | float  | Asset location, longitude.                         |

| Column          | Type  | Meaning                                                                   |
|-----------------|-------|---------------------------------------------------------------------------|
| age_is_inferred | bool  | Whether <code>asset_age</code> was inferred rather than sourced directly. |
| capacity_factor | float | Fraction of nameplate capacity the asset is expected to run at.           |
| emission_factor | float | Emissions per unit of output for this asset/technology.                   |

### companies\_ownerships.csv

Which companies own which assets, and to what degree. One row per asset-company ownership link per year.

| Column               | Type   | Meaning                                                 |
|----------------------|--------|---------------------------------------------------------|
| asset_id             | string | Unique identifier for the physical asset.               |
| company_id           | string | Unique identifier for the owning company.               |
| year                 | int    | Year this ownership link applies to.                    |
| ownership_percentage | float  | Share of the asset owned by the company, as a fraction. |
| sector               | string | Economic sector the asset belongs to.                   |
| technology           | string | Technology the asset uses within its sector.            |
| asset_name           | string | Human-readable name of the asset.                       |
| company_name         | string | Human-readable name of the owning company.              |

## scenarios.csv

Climate transition scenario pathways — projected prices, capacity, efficiency, and cost assumptions per scenario/technology/geography/year. These are the reference trajectories used to extend assets' and companies' production trajectories, and to calculate their forecasted profits.

| Column                   | Type                | Meaning                                                                               |
|--------------------------|---------------------|---------------------------------------------------------------------------------------|
| scenario                 | string              | Identifier of the climate scenario/pathway model run.                                 |
| scenario_type            | string              | Whether this scenario is used as the baseline or the target pathway.                  |
| scenario_geography       | string              | Geography the scenario pathway applies to.                                            |
| sector                   | string              | Economic sector the pathway applies to.                                               |
| technology               | string              | Technology the pathway applies to within its sector.                                  |
| year                     | int                 | Year this row's pathway values apply to.                                              |
| scenario_price           | float               | Output price projected under this scenario.                                           |
| fuel_price               | float               | Fuel input price projected under this scenario.                                       |
| scenario_pathway         | float               | Projected production/capacity pathway value for this row.                             |
| scenario_capacity_factor | float               | Fraction of nameplate capacity assumed under this scenario.                           |
| country_iso2_list        | string (comma-list) | ISO 3166-1 alpha-2 codes of countries covered by <a href="#">scenario_geography</a> . |
| lifetime_years           | float               | Assumed operating lifetime of the technology, in years.                               |

| Column                      | Type  | Meaning                                                     |
|-----------------------------|-------|-------------------------------------------------------------|
| efficiency_decimal          | float | Conversion efficiency of the technology, as a fraction.     |
| om_cost_usd_per_mw_per_year | float | Operations & maintenance cost per MW per year.              |
| capital_cost_usd_per_mw     | float | Capital expenditure per MW of capacity.                     |
| carbon_price_usd_per_tco2   | float | Carbon price assumed under this scenario, per tonne of CO2. |
| scrap_usd_per_mw            | float | Decommissioning scrap value per MW of capacity.             |

## 4. Pipeline reference

Each pipeline is namespaced under `conf/base/`, with its own inputs, outputs, and parameters file. Datasets are catalog entry names (`conf/base/catalog.yml`); parameters are keys from the pipeline's `parameters_<pipeline>.yml`, injected as `params:<key>`. The six pipelines below run in this order.

### 1. Inputs preparation

- **Pipeline:** `prepare_scenario_asset_and_company_inputs`
- **Inputs:** `assets_forecasts`, `companies_ownerships`, `scenarios`
- **Outputs:** `asset_forecast_panel`, `company_projection_inputs`

Turns the three raw input files into two model-ready tables. It filters the baseline/target scenario pathways, matches every asset to a scenario geography and to the CCS/non-CCS technology variant, and attaches each asset to its owning company.

- `asset_forecast_panel` is one row per physical asset per year (or, if `reduce_granularity_from_asset_to_company_level` is on, already collapsed to one row per company-technology).
- `company_projection_inputs` is one row per company/technology/geography/year, with the baseline and target scenario assumptions — price, cost, capacity factor, and so on — attached side by side, ready for the trajectory calculation in the next pipeline.

| Parameter                      | Type                       | Meaning and impact                                                                                                                   |
|--------------------------------|----------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| <code>baseline_scenario</code> | string                     | Scenario id used as the no-transition reference pathway that baseline trajectories follow.                                           |
| <code>target_scenario</code>   | string                     | Scenario id used as the transition pathway that shock trajectories move toward.                                                      |
| <code>company_ids</code>       | list of strings (optional) | Restricts the run to these companies; empty/unset processes all companies in the input data. Directly controls run size and runtime. |
| <code>ccs_on</code>            | bool or null               | Whether to use the with-CCS or without-CCS scenario variant for                                                                      |

| Parameter                                             | Type | Meaning and impact                                                                                                                                                                                                                                                                                                |
|-------------------------------------------------------|------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                       |      | Coal/Gas/Biomass technologies ( <b>null</b> = no distinction). Changes which technology rows are matched from <b>scenarios.csv</b> .                                                                                                                                                                              |
| <b>max_forecast_horizon</b>                           | int  | Number of years of forecast kept per asset/company. Larger = longer trajectories but more low-confidence out-years.                                                                                                                                                                                               |
| <b>reduce_granularity_from_asset_to_company_level</b> | bool | <b>True</b> aggregates inputs to one synthetic row per company-technology before the rest of the pipeline runs; <b>False</b> keeps real per-asset granularity. Changes the row count, identifiers, and structure of every downstream output — see <a href="#">Granularity changes the shape of every output</a> . |

## 2. Shock mechanism

- **Pipeline:** **calculate\_company\_trajectories**
- **Inputs:** **company\_projection\_inputs**
- **Outputs:** **company\_pathways\_pre\_allocation**

This is where the transition shock is defined at the company level. Every company/technology is classified into one of four cases — aligned or misaligned with its target pathway, crossed with whether the technology's output is increasing or decreasing under the scenario (e.g. renewables build-out vs. coal phase-down). Each case gets its own trajectory rule; the four are then combined into one path that follows baseline assumptions up to **shock\_year** and target assumptions from **shock\_year** through **alignment\_year** — the "late-sudden" transition shock.

- **company\_pathways\_pre\_allocation** has one row per company/geography/sector/technology/year/trajectory-type, where trajectory-type is **baseline**, **target**, or **late\_sudden\_requested** — the company-level path implied by the shock, before it's spread across individual assets in the next pipeline.

| Parameter                   | Type | Meaning and impact                                                                               |
|-----------------------------|------|--------------------------------------------------------------------------------------------------|
| <code>shock_year</code>     | int  | Year the late-sudden transition starts; assumptions switch from baseline to target in this year. |
| <code>alignment_year</code> | int  | Year by which requested trajectories reach the target pathway.                                   |

### 3. Staggered shock

- **Inputs:** `allocate_company_trajectories_to_assets`
- **Inputs:** `asset_forecast_panel`, `company_pathways_pre_allocation`
- **Outputs:** `asset_trajectories`, `company_trajectories`

Spreads the company-level shock down to individual assets — deciding, asset by asset, how much of a company's decline (or growth) each of its plants absorbs. Assets in decreasing technologies can retire on schedule and can have the shock staggered across them via a quantile curve rather than applied uniformly; assets in increasing technologies get topped up with a synthetic new-build asset for whatever capacity the company's real fleet can't deliver (see [Synthetic assets for increasing technologies](#)).

- `asset_trajectories` is one row per asset/year/trajectory-type and contains `baseline` and `latesudden`.
- `company_trajectories` adds `late_sudden_realized` on top of `baseline`, `target`, and `late_sudden_requested` — the sum of post-allocation asset capacity, so you can compare how much of the requested shock the asset fleet could actually absorb.

| Parameter                   | Type | Meaning and impact                                                           |
|-----------------------------|------|------------------------------------------------------------------------------|
| <code>shock_year</code>     | int  | Shared transition start year used when allocating company impacts to assets. |
| <code>alignment_year</code> | int  | Shared target-alignment year used by retirement and allocation logic.        |

| Parameter                                     | Type  | Meaning and impact                                                                                                     |
|-----------------------------------------------|-------|------------------------------------------------------------------------------------------------------------------------|
| <code>apply_retirement_baseline</code>        | bool  | Whether asset retirement dates are enforced in the baseline (no-shock) trajectory.                                     |
| <code>apply_retirement_shock</code>           | bool  | Whether asset retirement dates are enforced in the late-sudden (shock) trajectory.                                     |
| <code>apply_decreasing_staggered_shock</code> | bool  | Whether the shock is staggered across a quantile curve for technologies with decreasing output, vs. applied uniformly. |
| <code>staggered_shock.g_k</code>              | float | Curve-shape parameter for the staggered shock (steepness of the quantile allocation curve).                            |
| <code>staggered_shock.n_quantiles</code>      | int   | Number of quantile buckets the staggered shock is split across.                                                        |

#### 4. Earnings model

- **Pipeline:** `calculate_asset_earnings`
- **Inputs:** `asset_trajectories`
- **Outputs:** `asset_earnings`

Converts each asset's physical trajectory into money: revenue from output at the scenario price, fuel and carbon costs, fixed O&M, and CapEx (growth, replacement/roll-over, and decommissioning, each individually switchable), netting out to EBITDA and then free cash flow.

- `asset_earnings` is one row per asset/year/trajectory-type, with production, revenue, cost, EBITDA, and free-cash-flow columns, plus some identifying and classification columns used for the reporting.

| Parameter                       | Type        | Meaning and impact                                                                                                                              |
|---------------------------------|-------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>market_passthrough</code> | float (0-1) | Fraction of carbon price passed through to market prices. <code>0</code> = no passthrough, <code>1</code> = full passthrough; changes projected |

| Parameter                                | Type        | Meaning and impact                                                                                                                                               |
|------------------------------------------|-------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                          |             | revenue under the shock scenario.                                                                                                                                |
| <code>include_growth_capex</code>        | bool        | Whether to include CapEx for new-build capacity growth in the cost stack.                                                                                        |
| <code>include_replacement_capex</code>   | bool        | Whether to include CapEx for replacing retired eligible assets.                                                                                                  |
| <code>replacement_capex_rate</code>      | float (0-1) | Share of non-synthetic capacity growth capitalized as roll-over/replacement CapEx. Default <code>0.05</code> (5%).                                               |
| <code>include_decom_costs</code>         | bool        | Whether to include decommissioning costs for retired assets.                                                                                                     |
| <code>apply_continued_om_baseline</code> | bool        | Whether fixed O&M keeps being charged (at first-year capacity) after retirement, in the baseline trajectory. Only affects decreasing (high-carbon) technologies. |
| <code>apply_continued_om_shock</code>    | bool        | Same, for the shock trajectory. Only affects decreasing (high-carbon) technologies.                                                                              |

## 5. Valuation model

- **Pipeline:** `calculate_asset_and_company_npv`
- **Inputs:** `asset_earnings`
- **Outputs:** `asset_npv`, `company_npv`, `company_technology_npv`, `yearly_npv_trajectories`

Discounts each asset's free cash flow back to a present value — at one discount rate for baseline cash flows and a higher one for shock cash flows to reflect transition risk — optionally adding a terminal value beyond the forecast horizon, then rolls the result up from asset to company-technology to company level.

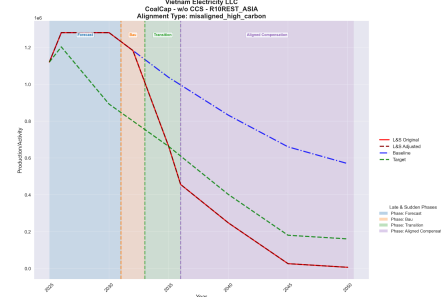
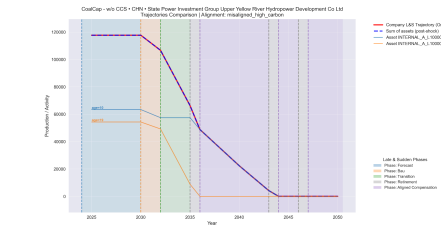
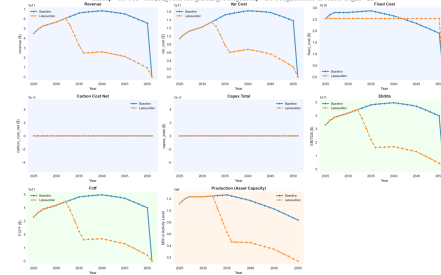
- `yearly_npv_trajectories` keeps the year-by-year discounted detail. `asset_npv` collapses that to one row per asset with `baseline_npv`, `latesudden_npv`, and the percentage `npv_change` between them.
- `company_technology_npv` and `company_npv` are the same comparison rolled up further, with an `asset_count` column showing how many assets (or synthetic technology buckets) sit behind each number.

| Parameter                                      | Type                            | Meaning and impact                                                                                                           |
|------------------------------------------------|---------------------------------|------------------------------------------------------------------------------------------------------------------------------|
| <code>dcf.discount_rate_baseline</code>        | float                           | Real discount rate applied to baseline cash flows in the DCF. Higher = lower NPV.                                            |
| <code>dcf.discount_rate_shock</code>           | float                           | Real discount rate applied to shock (late-sudden) cash flows; typically set higher than baseline to reflect transition risk. |
| <code>dcf.terminal_value.method</code>         | string ("none" or "perpetuity") | Whether a terminal value is added beyond the forecast horizon.                                                               |
| <code>dcf.terminal_value.g_real_default</code> | float                           | Real terminal growth rate used when <code>method</code> is "perpetuity".                                                     |

## 6. Reporting plots

- **Pipeline:** `plot_transition_risk_results`
- **Inputs:** `asset_earnings`, `company_trajectories`, `yearly_npv_trajectories`
- **Outputs:** `asset_financial_trajectories_plots_dir`

Turns the trajectory, earnings, and NPV tables above into the charts used to inspect transition-risk results.

| Plot                                                                                     | Folder                                        | Screenshot                                                                           |
|------------------------------------------------------------------------------------------|-----------------------------------------------|--------------------------------------------------------------------------------------|
| Company trajectories — baseline vs. target vs. realized activity per company/technology. | <code>companies_trajectories_plots/</code>    |   |
| Staggered shock — how the shock is distributed across a company's assets over time.      | <code>companies_staggered_shock_plots/</code> |   |
| Asset financial trajectories — revenue/cost/EBITDA/NPV detail per asset.                 | <code>asset_financial_trajectories/</code>    |  |

| Parameter                                               | Type | Meaning and impact                                             |
|---------------------------------------------------------|------|----------------------------------------------------------------|
| <code>plot_staggered_shock_use_log_scale</code>         | bool | Plots the staggered-shock chart's y-axis on a log scale.       |
| <code>plot_staggered_shock_show_shock_absorption</code> | bool | Overlays a shock-absorption band on the staggered-shock chart. |
| <code>reporting.plots.dpi</code>                        | int  | Resolution (dots per inch) plots are saved at.                 |

## 5. Kedro quickstart

ALTR is written as a [Kedro](#) project. The concepts that matter for using it:

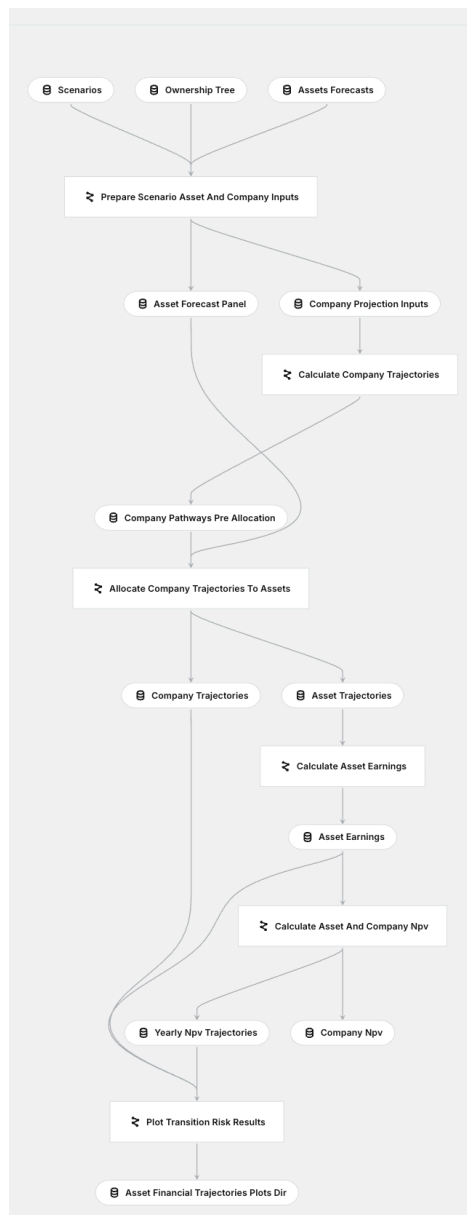
- **Nodes** are Python functions; **pipelines** wire nodes together.
- The **catalog** (`conf/base/catalog.yml`) declares where each dataset lives and how to read/write it. A catalog entry can point at a local CSV or a live source (a database, a warehouse table) — the pipeline code doesn't change either way, only the catalog entry does.
- **Parameters** (`conf/base/parameters*.yml`) are the model's tunable inputs, injected into nodes as `params:some_key`.
- **Environments** (`conf/<env>/`) let you override catalog/parameter entries per context. `conf/base/` is the shared default; `conf/local/` (gitignored) is for your own machine — personal overrides. Kedro merges `base` with whichever environment you pass via `--env`.

**The model doesn't depend on Kedro to run.** Nodes are plain Python functions (`pipelines/*/nodes.py`) that take DataFrames/params in and return DataFrames out — Kedro's role is orchestration (wiring, the catalog, `kedro run`), not the modeling logic itself. If you ever want to drop the framework, the node functions can be called directly from a script or notebook with plain pandas DataFrames; nothing in the modeling code itself is Kedro-specific.

Using the package kedro-viz, it is possible to generate a graph of the model pipeline.

```
Shell  
kedro viz
```

Opens a web UI showing the pipeline graph, node inputs/outputs, and data lineage.



## 7. Additional notes

### Conditions a scenario has to respect to be economically viable

Not every `scenario` × `technology` × `geography` combination in `scenarios.csv` is economically sane — some combinations imply an asset can never turn a profit under that scenario, no matter how the run parameters are set. Two quick checks worth running against `scenarios.csv` before picking a `baseline_scenario/target_scenario` pair:

- **Fixed cost check.** If `om_cost_usd_per_mw_per_yr` is higher than `scenario_capacity_factor` × 8,760 hours × `scenario_price` — i.e. the fixed O&M bill alone exceeds what the asset could earn running at its scenario capacity factor for a full year — EBITDA is guaranteed negative for that row regardless of fuel or carbon costs.
- **Variable cost check.** If `fuel_price` ÷ `efficiency_decimal` is higher than `scenario_price` — i.e. the fuel cost of producing one more unit of output already exceeds the price received for it — every unit produced loses money before fixed costs are even considered.

Either condition flags a scenario/technology combination the model will dutifully run the numbers on, but that has no realistic economic future — worth a sanity check on your input data rather than a debugging session on the output.

### How assets get matched to a scenario geography

Every asset is assigned exactly one `scenario_geography` (used as a merge/groupby key for the rest of the pipeline), via `country_iso2` against the geography → country mapping implied by `country_iso2_list` in `scenarios.csv`.

The most specific match wins: if a country is covered by more than one scenario geography (e.g. a single-country entry *and* a multi-country regional bucket), the geography with the *fewest* countries is used — an exact-country geography beats a 10-country region, which beats a 100-country region. If two geographies cover the same country with the same specificity, the run fails rather than picking one arbitrarily.

### Granularity changes the shape of every output

The `reduce_granularity_from_asset_to_company_level` parameter (see [Pipeline reference](#)) changes what an "asset" is in the output, not just how many rows there are:

1. **Asset granularity** (parameter set to `False`): `company_npv.csv`'s `asset_count` reflects the true number of physical assets owned by the company.

2. **Company granularity** (parameter set to `True`): physical assets are aggregated into one synthetic row per company/sector/technology/geography before the model runs. `asset_npv.csv`'s `asset_id` becomes a generated id like `NEW_CN_2023551807935057693_Power_HydroCap_EU`, and `company_npv.csv`'s `asset_count` drops to the number of technology/geography buckets rather than physical assets.

Don't mix outputs from different granularity settings when comparing runs — aggregating before running the model is not equivalent to aggregating the model's asset-level output afterwards, since retirement, staggered-shock, and continued-O&M logic all operate differently on synthetic technology-level "assets" than on real ones.

### Asset retirement age: refurbishment wrap-around, not a hard cutoff

An asset's observed age is not used as-is. At its first valid observation, age is wrapped modulo the technology's `lifetime_years`, then ages linearly from there — i.e. the model assumes assets are refurbished on a rolling lifetime cycle rather than permanently retired the first time they exceed their nominal lifetime. This changes which assets the model treats as "old" (near their *next* retirement point) versus "recently refurbished," and therefore which assets are subject to retirement-driven capacity drop-off under `apply_retirement_baseline/apply_retirement_shock`.

### Synthetic assets for increasing technologies

For technologies whose scenario pathway is *increasing* (e.g. renewables build-out), a company's real assets are left at business-as-usual capacity, and any gap versus the company's target trajectory is filled by a single **synthetic** top-up asset.

**A synthetic asset can only appear where the company already has a real one.** This is because the model only produces a company trajectory for (`company`, `scenario_geography`, `sector`, `technology`) combinations where the company already has at least one real asset there. A company can never get a synthetic asset in a country/technology it has no real presence in at all.

From `shock_year` onward, the synthetic asset's capacity in year  $t$  is `max(0, company_target[t] - sum(real_assets[t]))` — it fills exactly the gap between the company-level target and what real assets already deliver, so real + synthetic always reconciles to the company total.